



Adriana Rocha
André Silva
Dinis Oliveira
Guilherme Goulart
Rafael Ferreira

Modular Monitoring and Performance
Benchmarking in Dynamic Datacenters



Modular Monitoring and Performance Benchmarking in Dynamic Datacenters

Este projeto foi concebido e desenvolvido no âmbito da Unidade Curricular de Projeto em Engenharia de Computadores e Informática, realizada sob a orientação científica de Professor Doutor Daniel Nunes Corujo, Professor associado do Departamento de Eletrónica, Telecomunicações e Informática da Universidade de Aveiro, e de Professor Doutor Rui Luis Andrade Aguiar, Professor catedrático do Departamento de Eletrónica, Telecomunicações e Informática da Universidade de Aveiro, e sob a contribuição de Alisson Chaves, David Santos, Tiago Barros e Rui Silva.

orientador / professor

Prof. Doutor Daniel Corujo

professor associado da Universidade de Aveiro

orientador / professor

Prof. Doutor Rui Luis Andrade Aguiar

professor catedrático da Universidade de Aveiro

keywords

virtualisation, observability, orchestration, stress test, benchmarking

abstract

This report presents a unified platform for infrastructure observability and performance testing, developed in collaboration with Instituto de Telecomunicações. The system integrates multi-source metrics collection, controlled stress testing, and hardware benchmarking into a single extensible framework, producing structured and labelled data as the foundation for future intelligent analysis. A proof-of-concept implementation demonstrates the platform's ability to collect 28 metrics across compute, energy, and electrical sources via dynamically scheduled Telegraf pods, execute parameterised stress and benchmark workloads on virtualised nodes through Argo Workflows, and expose results through a unified dashboard. The modular design — where adding a new collection source or test module requires only a container image and a WorkflowTemplate definition — enables the platform to scale to new infrastructure without architectural changes. The accumulated structured run history positions the framework as a basis for future ML-driven intelligent performance analysis.

acknowledgement of use of AI tools

Recognition of the use of generative Artificial Intelligence technologies and tools, software and other support tools.

The development of this work was supported by several artificial intelligence tools, which were used to assist with research, writing, and architectural reasoning.

Claude (Anthropic) was used extensively throughout the project for architectural design discussions, LaTeX drafting, and iterative refinement of technical content.

ChatGPT (OpenAI) was consulted for complementary perspectives on technical design decisions and for clarification of concepts during the research phase.

Consensus is an AI-powered academic search engine that was used to locate and surface peer-reviewed literature relevant to the topics of infrastructure benchmarking, anomaly detection, and time-series analysis. It supported the identification of primary sources cited throughout this work.

In all cases, the authors retain full responsibility for the content, accuracy, and conclusions presented in this report. AI-generated suggestions were critically reviewed and adapted before incorporation; no section was included without author verification.

Contents

Contents	i
List of Figures	ii
List of Tables	iii
1 Introduction	1
1.1 Context	1
1.2 Problem	1
1.3 Goals	2
1.4 Methodology	2
1.5 Report Structure	3
2 State of the Art	4
2.1 Related Work	4
2.2 Tools and Technologies	6
3 Requirements	8
3.1 Requirements Gathering	8
3.2 Functional Requirements	8
3.3 Non-Functional Requirements	9
3.4 Actors and Personas	10
3.5 Use Cases	10
4 Architecture	13
4.1 Metrics Collection Architecture	13
4.2 Stress Testing and Performance Benchmarking Architecture	18
4.3 User Interface Architecture	22
4.4 Machine Learning Pipeline — Exploratory Architecture	24
5 Results	26
5.1 MVP - Minimum Viable Product	26
5.2 Observability and Monitoring Capabilities	26
5.3 Testing and Validation Framework	27
5.4 Dashboard	27
5.5 Documentation and Deliverables	33
6 Conclusions & Future Work	34
6.1 Conclusions	34
6.2 Future Work	35
Bibliography	37

List of Figures

4.1	Metrics collection architecture. Left: the Collector Node hosting Argo Workflow, dynamically scheduled Telegraf pods, InfluxDB, Grafana, and Kafka. Right: the monitored datacenter infrastructure comprising multiple physical hosts, each running a hypervisor with one or more virtual machines.	13
4.2	Stress testing architecture showing the Test Stress Manager (master) and the VM Under Test (worker), connected through Argo Workflow scheduling and k6 HTTP load generation. . . .	19
4.3	Overall system architecture showing the UI, the stress testing cluster (TESTS), and the metrics collection cluster (COLLECTOR), with the four typed communication channels between them.	22
4.4	High-level ML pipeline overview. Raw metrics from InfluxDB and Kafka are transformed into a shared feature store, used to train three model families, and served to a Grafana dashboard via batch and streaming inference paths.	24
4.5	Inference paths. The streaming lane produces near-real-time anomaly alerts; the batch lane produces degradation forecasts and bottleneck reports. Both write results to InfluxDB for visualisation in Grafana.	25
5.1	Home page	27
5.2	Collector page	28
5.3	Stress Tests page	29
5.4	Benchmarking page	31
5.5	VM Dashboard page	32

List of Tables

3.1	Functional Requirements - Monitoring System.	9
3.2	Non-Functional Requirements - Monitoring System.	9
4.1	KPIs collected from the Proxmox source.	14
4.2	KPIs collected from the Scaphandre source.	15
4.3	KPIs collected from the PDU source.	15
4.4	Collector Node component summary.	18
4.5	Infrastructure stress pod definitions.	20
4.6	Benchmark modules and their target subsystems.	22
4.7	UI communication channels.	23
4.8	Proposed models per ML task.	24
5.1	Collector configuration parameters.	28
5.2	Stress test input parameters.	30
5.3	Benchmark input parameters.	32

Introduction

With the continuous rapid growth of digital services and cloud computation, datacenters have been forced to innovate the way they are projected and operated.

Nowadays, these environments are constantly getting more dynamic, already providing support for distributed, scalable applications developed on modern architectures.

Basing these systems on virtualization, containerized workloads, software-defined networking (SDN) and emphasized focus on microservices. These approaches allow more flexibility, elasticity, and better efficiency in resource utilization, granting the sustained scalability and adaptability of services demanded by the system.

As a result, the complexity of these systems keeps growing, and new challenges arise. Monitoring, performance management and reliability are a pressing concern.

1.1 Context

In this project, our focus is on monitoring and performance management in modern data-center environments. We are working with the Telecommunications Institute (IT) department of the University of Aveiro. Their focus is on investigation with strong activities in the network, distributed systems and computer infrastructures branches. The IT of Aveiro collaborates regularly with other academic and industrial partners, contributing for the development of innovative solutions in real-world scenarios, with demanding requirements in performance and reliability.

1.2 Problem

Traditional monitoring solutions present various limitations when applied to these highly dynamic environments. In such environments, maintaining performance stability and system reliability has become a significant challenge. Performance degradation can arise from multiple sources, including resource contention, workload variability, and cascading failures between dependent services.

Furthermore, the complexity of multi-layer architectures—spanning applications, operating systems, networking components, and underlying hardware—makes it harder to perform a comprehensive performance analysis and get a clear view of the system's behavior. A further drawback of legacy systems is their reactive nature, as they only identify failures after performance has been impacted.

A critical aspect, frequently neglected is the lack of integration between stress testing and fault injection systems and monitoring systems, making it harder to evaluate the resilience of the system and analyze the response to adversities. The insufficiency in environments with continuous and automated testing limit the capacity to predict failures and validate mitigation strategies.

These constraints, fuel the need for new, more advanced approaches not only allowing more automation, easier monitoring but also comprehensive testing in a single framework.

1.3 Goals

The work presented in this report proposes and implements an integrated monitoring and control framework for dynamic datacenter environments. The system is structured around a React-based user interface that serves as the single point of human interaction, bridging two independent backend subsystems: a Collector Node that orchestrates multi-source metric collection via dynamically scheduled Telegraf pods, persisting data in InfluxDB and streaming selected metrics through Kafka; and a Stress Testing cluster, coordinated by Argo Workflows, that enables controlled fault injection and performance benchmarking on target virtual machines. Together, these components form a proof-of-concept platform developed in collaboration with the Telecommunications Institute of Aveiro, targeting the real experimental infrastructure managed by the institute.

The proposed approach seeks to:

- Improve observability through autonomous multi-layer metric collection and analysis;
- Support automated mechanism for stress testing and fault injection, enabling analysis and mitigating performance degradation;
- Ease early detection of anomalies, making the system more proactive;
- Apply ML techniques for anomaly detection, performance prediction and bottleneck identification;
- Provide a dashboard for visualisation and interaction.

The expected outcome of this work is a functional prototype delivering a unified dashboard through which an operator can configure and trigger KPI collection from virtual machines, execute stress tests and benchmarking workflows on target infrastructure, and visualise the collected metrics in real time. The platform aims to demonstrate that these three capabilities — observability, stress testing, and benchmarking — can be effectively integrated into a single, cohesive tool for dynamic datacenter environments.

1.4 Methodology

The development of this platform followed an iterative, component-based agile methodology. The system was built incrementally, with each subsystem developed and validated independently before being integrated into the full platform. Work was divided across the team by subsystem — metrics collection, stress testing, and user interface — allowing parallel development streams to progress independently. Weekly feedback sessions were held with the project supervisors and contributors, who are also the primary users of the platform and therefore provided direct insight into operational needs, guiding priorities, validating design decisions, and refining requirements throughout the development cycle.

1.5 Report Structure

Chapter 2 reviews the state of the art in datacenter observability, chaos engineering, and intelligent infrastructure analysis, contextualising the work within related European research projects. Chapter 3 defines the functional and non-functional requirements of the system, together with the actor personas and use cases that guided its design. Chapter 4 describes the architecture of the three main subsystems — metrics collection, stress testing, and user interface — as well as an exploratory machine learning pipeline. Chapter 5 presents the final results. Finally, Chapter 6 concludes the report and outlines directions for future work.

State of the Art

2.1 Related Work

The growing adoption of cloud-native and microservices architectures in datacenter environments has driven intense research in the areas of observability, chaos engineering, and intelligent infrastructure analysis. This section reviews the most relevant work organized around the three main pillars of the proposed system—dynamic observability, fault injection, and automated analysis—and contextualizes this work within the scope of European research projects with which it shares infrastructure and objectives.

2.1.1 European Associated Projects

The three European research projects described below share a common operational challenge: they all rely on the same experimental computing infrastructure managed at the Telecommunications Institute of Aveiro, and all require that infrastructure to be observable, resilient, and testable under realistic conditions. The monitoring and control framework developed in this work directly addresses that shared need, providing the data collection, stress testing, and benchmarking capabilities that each project requires to validate its objectives.

2.1.1.1 IMAGINE-B5G [1]

It's a project which aims to create an advanced and accessible end-to-end Beyond-5G (B5G) communications platform for trials and large-scale pilots. The platform brings together four experimental facilities in Portugal, Spain, Norway, and France, supporting the development of advanced applications in sectors such as healthcare, transportation, smart agriculture, and Industry 4.0.

From the perspective of this work, IMAGINE-B5G represents a concrete operational context: the underlying experimental infrastructure must remain observable and performant while B5G trials are actively running. The framework developed here provides the KPI collection and stress testing mechanisms that enable the institute's team to validate infrastructure behaviour without disrupting ongoing trials.

2.1.1.2 EXIGENCE [2]

It's a project focused on reducing the energy consumption and carbon footprint of ICT service delivery across heterogeneous ecosystems. It operates on three pillars: measuring energy consumption at the service level across multiple domains, optimizing resource provisioning, and fairly attributing energy responsibilities to service consumers. The experimental computing infrastructure managed by the supervisors of this work is one of the EXIGENCE testbeds, where fine-grained monitoring is required to validate the project's energy efficiency models.

For this work, EXIGENCE introduces a specific monitoring requirement that goes beyond compute metrics: energy consumption must be measured at fine granularity, attributed to individual virtual machines and processes, and correlated with service-level activity. The integration of Scaphandre-based energy collection into the framework directly responds to this requirement, enabling the testbed to produce the energy attribution data that EXIGENCE's models depend on.

2.1.1.3 6G-VERSUS [3]

It's a project that explores the frontiers of 6G technology through trials and pilots across six European countries, including Portugal, assessing the social, economic, and environmental impact of next-generation technologies. The project introduces a methodology combining Vertical Applications (V-App), Network Applications (N-App), and AI-assisted Applications (AI-App), supported by distributed 6G platforms across Europe. The ability to dynamically monitor the underlying infrastructure of these trials — adapting to topology changes without manual intervention — is a direct operational need that this work addresses.

For this work, 6G-VERSUS motivates the dynamic adaptability of the collection architecture: as trial topologies change, the monitoring system must follow without manual reconfiguration. The use of dynamically scheduled Telegraf pods orchestrated by Argo Workflows — rather than statically configured agents — is a direct design response to this operational requirement.

Across these three projects, a common operational requirement emerges: the shared infrastructure at IT Aveiro must support observability, energy monitoring, and resilience testing concurrently, adapting to topology changes without manual intervention. The framework developed in this work addresses precisely this convergence of requirements.

2.1.2 Observability in Cloud-Native Environments

Several surveys on observability in cloud-native applications conclude that tool fragmentation and heterogeneous telemetry formats are the main obstacles to end-to-end visibility, identifying metrics, logs, and traces as its three fundamental pillars. Projects such as OpenTelemetry and Prometheus have since converged toward a unified instrumentation approach, which is particularly relevant given the heterogeneous components of the shared infrastructure across IMAGINE-B5G, EXIGENCE, and 6G-VERSUS.

More focused studies demonstrated that redefining collection targets at runtime, without system restart, is key to accommodating dynamic infrastructure changes, and that combining Prometheus with Grafana in Kubernetes environments significantly reduces anomaly detection time. Both principles are directly applicable to this project, where infrastructure components may be added or removed without requiring changes to the monitoring system itself.

2.1.3 Chaos Engineering and Fault Injection

A reference framework for integrating fault injection into continuous delivery pipelines introduced metrics such as Mean Time to Detect (MTTD) and Mean Time to Recovery (MTTR) to quantify resilience, both of which have been adopted in this project. Further research argued that ignoring inter-component dependencies when selecting fault injection points leads to superficial resilience assessments, which is a relevant concern given the complex dependencies between network, compute, and orchestration layers in the shared experimental infrastructure. Additionally, a significant gap in application-level fault coverage compared to infrastructure-level faults has been identified, motivating the inclusion in this project of fault injection capabilities at both levels.

2.1.4 Intelligent Analysis and Anomaly Detection

Time-series-based anomaly detection has been shown to outperform threshold-based alerting under variable load, reducing false positives without increasing detection latency. This is complemented by PromQL-native anomaly detection frameworks using sliding time windows, avoiding dependency on external machine learning systems. Recent research additionally explores the use of Large Language Models such as Llama and Mistral for automated Root Cause Analysis from observability data, which motivates the intelligent analysis component of this project.

2.2 Tools and Technologies

Building the framework described in this report requires solving four distinct engineering problems: virtualising and orchestrating the infrastructure under observation; collecting metrics from heterogeneous sources without intruding on monitored workloads; storing, streaming, and visualising those metrics at scale; and executing controlled stress tests and benchmarks in a repeatable, parameterised manner.

This section introduces the tools selected to address each of these problems. Their detailed configuration and integration within the system architecture are described in Chapter 4.

- **Virtualisation and orchestration** - Proxmox VE[4] serves as the physical hypervisor layer, managing the virtual machines that form both the monitored infrastructure and the system's own components. Kubernetes[5] handles container orchestration across the collector and stress testing nodes, with k3s[6] adopted as a lightweight distribution suited to the constrained footprint of the Collector Node. Argo Workflows[7] provides the workflow engine that schedules and parameterises both Telegraf collection pods and stress test executions, enabling dynamic, on-demand pipeline execution without static agent configuration.
- **Observability and metric collection** - Telegraf[8] serves as the collection agent, deployed as dynamically scheduled pods for each metric source. InfluxDB[9] provides the time-series storage layer, persisting tagged metrics with configurable retention policies. Grafana[10] connects to InfluxDB as its primary data source and provides the operator-facing visualisation layer, including run-correlated annotations that align stress test timelines with collected metrics. Kafka[11] adds an optional streaming bus for real-time alerting and external data consumption. Scaphandre[12] enables energy consumption measurement at the process and VM level, addressing the fine-grained energy attribution requirement introduced by EXIGENCE.

- **Stress testing and benchmarking** - The stress module was developed as an adaptation of a previous module already constructed by IT Aveiro researchers for a niche usecase. Our module provides containerised infrastructure stress workloads targeting CPU, memory, disk I/O, and network resources. Chaos Mesh[13] handles Kubernetes-native fault injection. Iperf3[14] is used for network load generation and as a network performance benchmark. fio[15] measures storage I/O performance across sequential and random access patterns. k6 generates realistic HTTP load against the QuickPizza[16] application deployed on the worker node, enabling application-level degradation measurement under resource contention.

Requirements

3.1 Requirements Gathering

The system requirements were gathered primarily through brainstorming sessions with the project supervisors and contributors, who also represent end-users of the datacenter monitoring platform. These discussions made it possible to identify the key functionalities, objectives, and constraints of the platform, with particular emphasis on datacenter infrastructure monitoring, energy consumption, and system management.

The platform to be built is a unified dashboard through which an operator can configure and trigger KPI collection from virtual machines, execute stress tests and benchmarking workflows on target infrastructure, and visualise the collected metrics in real time. The system is structured around a React-based user interface that serves as the single point of human interaction, bridging two independent backend subsystems: a Collector Node responsible for orchestrating multi-source metric collection, and a Stress Testing cluster that enables controlled fault injection and performance benchmarking. The platform is deployed and validated on the real experimental infrastructure managed by the Telecommunications Institute of Aveiro.

3.2 Functional Requirements

The functional requirements define the concrete capabilities the system must provide in order to fulfil its monitoring, testing, and analysis objectives.

Reference	Functional Requirements
FR-1	Collect infrastructure key performance indicators (KPIs) from virtual machines in a non-intrusive manner;
FR-2	Centralise all monitoring data in a unified storage layer, enabling consistent access for analysis and historical queries;
FR-3	Provide mechanisms to perform controlled fault injection and stress testing scenarios (e.g. CPU exhaustion, network latency) on target infrastructure;
FR-4	Analyse collected data using machine learning and statistical algorithms to identify behavioural deviations and saturation trends;
FR-5	Provide customisable dashboards and configurable alerts for infrastructure operators, enabling real-time visibility and rapid incident response;

Table 3.1: Functional Requirements - Monitoring System.

3.3 Non-Functional Requirements

Beyond functional behaviour, the system must satisfy a set of quality attributes that govern its operational characteristics. These non-functional requirements are described below.

Reference	Non-Functional Requirements
NFR-1	The system must handle large volumes of metrics and logs without data loss, supporting a growing number of monitoring sources;
NFR-2	Data collection and fault injection components must support horizontal scaling through Kubernetes, allowing the platform to grow with the monitored infrastructure;
NFR-3	Monitoring agents must have a minimal footprint on the resources of monitored machines, ensuring that the act of observation does not significantly affect the observed system;
NFR-4	The observability pipeline must be resilient to partial failures, maintaining continuous data collection even when individual components are degraded;
NFR-5	The system must detect anomalies within a few seconds of their occurrence, enabling timely response to critical infrastructure events;
NFR-6	Machine learning-based anomaly detection must achieve high precision and recall, minimising both false positives and missed detections;

Table 3.2: Non-Functional Requirements - Monitoring System.

3.4 Actors and Personas

Two primary actor personas were defined to represent the target users of the system. These personas guided use-case design and helped prioritise features according to real operational needs.

3.4.1 Persona 1 — System Administrator

“My goal is to ensure overall infrastructure health and optimise resource allocation.”

Profile: Responsible for maintaining server and network stability throughout the datacenter.

Key Needs and Actions:

- *Customised Analysis* — Conducts performance audits of specific network segments or VM clusters.
- *KPI Selection* — Defines critical metrics such as network throughput, disk IOPS, and CPU saturation.
- *Capacity Planning* — Uses historical time-series data (stored in InfluxDB) to identify growth trends and anticipate resource requirements.
- *Energy Efficiency* — Monitors the impact of energy efficiency policies through the system’s dedicated dashboard module.

3.4.2 Persona 2 — Test Engineer

“My goal is to validate the performance and behaviour of the service I am developing or running.”

Profile: Focused on the software lifecycle and validation of microservices in dynamic environments.

Key Needs and Actions:

- *Performance Benchmarking* — Analyses service behaviour under peak load by selecting application latency and error-rate KPIs.
- *Fault Injection* — Triggers stress scenarios (via Chaos Mesh or stress-ng) on specific VMs to test auto-healing mechanisms.
- *Real-Time Analysis* — Uses Kafka streaming to observe service degradation at the precise moment a fault is injected.
- *SLA Validation* — Compares observed results against predefined performance limits (SLAs) established for the project.

3.5 Use Cases

Six primary use cases were identified based on the personas and functional requirements. The individual use cases are described in the following subsections.

3.5.1 UC1 - Metric Collection Configuration and Execution

Description: The operator configures a collection session by selecting the desired metric sources, KPIs, target virtual machines and duration through the UI. Once configured, the operator submits the collection request, which triggers the scheduling of the appropriate Telegraf pods on the Collector Node.

Objective: Enable the operator to define and launch a targeted, parameterised metric collection session without manual intervention on the underlying infrastructure.

Primary Actor: System Administrator (Persona 1)

Precondition: The Collector Node is operational and the target virtual machines are accessible.

Main Flow: Operator selects metric sources, KPIs, target VMs, and duration, then submits Collector Environment Set and Collector Request via the UI. Argo Workflows schedules the corresponding Telegraf pods and metrics begin flowing to InfluxDB.

3.5.2 UC2 — Real-Time Telemetry Processing

Description: Collected metrics are injected into a streaming pipeline via Apache Kafka.

Objective: Ensure that the analysis engine receives data with low latency, enabling an immediate response to critical events.

Primary Actor: System (automated pipeline)

Precondition: Monitoring agents are deployed and actively emitting metrics.

Main Flow: Agents publish metric payloads to Kafka topics → consumers in the analysis engine read and process the stream in near real-time.

3.5.3 UC3 — Multi-Layer Visualisation and Monitoring

Description: The operator accesses a centralised Grafana dashboard to visualise real-time metrics from hardware, network, and container layers.

Objective: Provide a holistic view of datacenter health and enable visual detection of performance variations.

Primary Actor: System Administrator (Persona 1)

Precondition: Data is being collected and stored; Grafana data sources are configured.

Main Flow: Operator opens dashboard → selects time range and target infrastructure → inspects panels for anomalies or trends.

3.5.4 UC4 — Data Persistence and Dataset Generation

Description: The system automatically stores normalised metrics in InfluxDB (a time-series database).

Objective: Enable historical analysis and structured dataset export for machine learning model training.

Primary Actor: System (automated pipeline); Test Engineer (dataset export)

Precondition: Metrics are flowing through the pipeline; InfluxDB is operational.

Main Flow: Metrics are written to InfluxDB with timestamps → user queries or exports data for a given time window → dataset is generated in the required format.

3.5.5 UC5 — Resilience Validation via Fault Injection

Description: The user schedules controlled fault introduction (e.g. network latency, CPU exhaustion) using tools such as Chaos Mesh or stress-ng.

Objective: Assess infrastructure robustness and evaluate the impact of cascading failures on services.

Primary Actor: Test Engineer (Persona 2)

Precondition: Target VMs or containers are reachable; fault injection tools are configured.

Main Flow: Engineer defines fault scenario and target → activates injection → monitors system response in real-time through UC2 → compares results against SLAs.

3.5.6 UC6 — Intelligent Analysis and Anomaly Detection

Description: The system applies ML and statistical algorithms to the data stream to identify behavioural deviations and saturation trends.

Objective: Transition from reactive monitoring to a proactive posture, detecting issues such as memory leaks before they affect end users.

Primary Actor: System (automated); System Administrator (alert recipient)

Precondition: Sufficient historical data is available for model training; models are deployed.

Main Flow: Incoming metric stream is evaluated against trained models → deviations exceeding thresholds trigger alerts → operator is notified via the dashboard or alerting channel.

Architecture

4.1 Metrics Collection Architecture

The metrics collection subsystem is responsible for gathering, tagging, and routing performance indicators and key performance indices (KPIs) from the monitored datacenter infrastructure. It is designed around a single *Collector Node* — a virtual machine running on the same Proxmox host as the VMs under observation — which centralises all collection logic and exposes a uniform data path to storage, visualisation, and streaming outputs. Figure 4.1 shows the full topology.

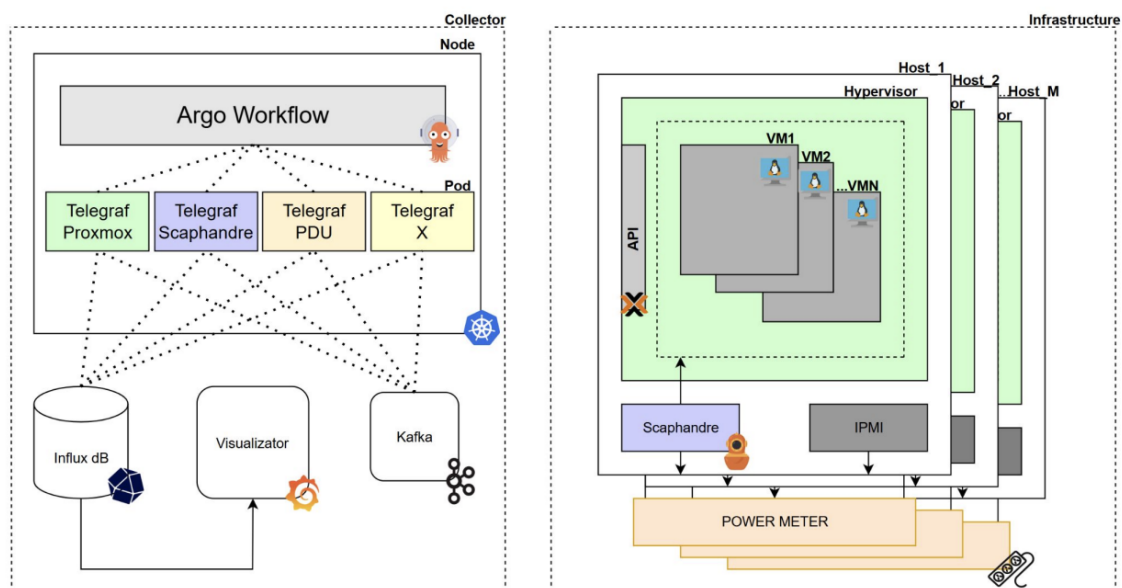


Figure 4.1: Metrics collection architecture. Left: the Collector Node hosting Argo Workflow, dynamically scheduled Telegraf pods, InfluxDB, Grafana, and Kafka. Right: the monitored datacenter infrastructure comprising multiple physical hosts, each running a hypervisor with one or more virtual machines.

4.1.1 Infrastructure Being Monitored

The right-hand side of Figure 4.1 represents the datacenter infrastructure whose metrics are to be collected. It consists of M physical hosts (Host₁ through Host_M), each running a hypervisor layer. In the reference implementation the hypervisor is Proxmox VE [4], which exposes a RESTful management API and hosts an arbitrary number of virtual machines (VM₁ through VM_N).

The architecture is explicitly designed to be hypervisor-agnostic. The Proxmox API endpoint, represented by the API marker in the diagram, is the integration point that Telegraf targets; replacing it with an equivalent endpoint from another virtualisation platform — such as OpenStack [17] — requires only a change in the Telegraf input plugin configuration, with no modification to the surrounding orchestration or storage infrastructure. This extensibility is a deliberate design goal, enabling the same collector stack to operate across heterogeneous datacenter environments.

Prerequisites and operational boundaries. The collection architecture assumes that the physical infrastructure — PDUs, hosts, and any network-attached devices — is already operational and accessible from the Collector Node. Configuration of the underlying hardware, including PDU SNMP community strings, Scaphandre daemon deployment on physical hosts, and Proxmox API credentials, falls within the responsibility of the datacenter administrator. The collector subsystem consumes these interfaces as-given; its role is limited to scheduled ingestion, tagging, and routing of the metrics they expose, not to the provisioning or management of the infrastructure itself.

4.1.2 Key Performance Indicators

Metrics are organised by collection source. Each source is served by a dedicated Telegraf pod and produces a distinct set of KPIs that together characterise the computational, energetic, and electrical state of the monitored infrastructure.

4.1.2.1 Proxmox — Virtual Machine and Host Metrics

The Proxmox input plugin collects per-VM and per-host statistics directly from the Proxmox VE REST API without requiring any agent inside the guest VMs. Metrics cover compute utilisation, memory consumption, disk and network throughput, and VM lifecycle state.

Table 4.1: KPIs collected from the Proxmox source.

KPI	Unit	Description
cpu	%	CPU utilisation of the VM
cpus	count	Number of virtual CPUs allocated
mem	bytes	Current memory in use
maxmem	bytes	Memory allocated to the VM
disk	bytes	Disk space in use
maxdisk	bytes	Total disk space allocated
netin	bytes/s	Network ingress throughput
netout	bytes/s	Network egress throughput
diskread	bytes/s	Disk read throughput
diskwrite	bytes/s	Disk write throughput
uptime	seconds	Time since last VM boot
status	categorical	VM running state (running, stopped, ...)

4.1.2.2 Scaphandre — Power and Energy Metrics

Scaphandre [12] reads from hardware power interfaces such as Intel RAPL and IPMI to produce fine-grained energy metrics at host, socket, power domain, and process level. This granularity allows power consumption to be attributed not only to physical hosts but to individual virtual machines and processes running within them, enabling energy-aware performance analysis alongside the compute metrics collected from Proxmox.

Table 4.2: KPIs collected from the Scaphandre source.

KPI	Unit	Description
host-power	W	Instantaneous power draw of the host
host-energy	J	Cumulative energy consumed by the host
socket-power	W	Power draw per CPU socket
socket-energy	J	Cumulative energy per CPU socket
domain-power	W	Power draw per RAPL power domain
domain-energy	J	Cumulative energy per RAPL domain
proc-power	W	Estimated power attributed to a process
proc-cpu	%	CPU utilisation of the process
proc-mem	bytes	Resident memory of the process
proc-mem-virt	bytes	Virtual memory of the process
proc-disk-r	bytes/s	Current disk read rate of the process
proc-disk-r-total	bytes	Cumulative bytes read by the process
proc-disk-w	bytes/s	Current disk write rate of the process
proc-disk-w-total	bytes	Cumulative bytes written by the process

4.1.2.3 PDU — Electrical Power Metrics

The PDU Telegraf pod collects electrical power measurements from rack-mounted Power Distribution Units via SNMP. Two metrics are exposed: per-outlet active power and aggregate rack-level power draw. These readings complement Scaphandre’s software-derived power estimates with ground-truth electrical measurements at the infrastructure boundary, enabling validation of energy attribution and total rack power budgeting.

Table 4.3: KPIs collected from the PDU source.

KPI	Unit	Description
outlet-power	W	Active power draw on a specific PDU outlet
total-power	W	Aggregate power draw across all PDU outlets

4.1.3 Collector Node

The Collector Node is a virtual machine provisioned on the same Proxmox host as the infrastructure it monitors. It runs a lightweight Kubernetes distribution (k3s [6]), chosen specifically because its minimal footprint makes it straightforward to bring the entire collection stack up or down as a unit, and because individual collection components can be added, removed, or reconfigured as Kubernetes pods without disrupting the rest of the system.

Four logical components run on this node, described in the subsections that follow: Argo Workflows for orchestration, Telegraf pods for collection, InfluxDB and Grafana for storage and visualisation, and Kafka as an optional streaming bus.

4.1.4 Argo Workflow Orchestration

Argo Workflows serves as the orchestrator for the entire collection layer, mirroring its role in the stress testing subsystem (Section 4.2). When a monitoring session is initiated from the user interface, Argo receives the selected collection profile together with its arguments — target hosts, polling intervals, enabled metric sources, and output destinations — and schedules the corresponding Telegraf pods on the Collector Node.

This design makes the collection topology fully dynamic. A lightweight datacenter with few hosts requires only a small number of Telegraf pods; a larger environment can scale horizontally by submitting a workflow with a higher parallelism parameter. Pods that are no longer needed are terminated cleanly by Argo at the end of the workflow, freeing resources on the Collector Node. The dashed lines in Figure 4.1 represent this many-to-many scheduling relationship between the Argo controller and the pool of active Telegraf pods.

4.1.5 Telegraf Collection Pods

Each Telegraf instance runs as an independent Kubernetes pod and is responsible for collecting metrics from one logical source. Four pod types are shown in the diagram:

- **Telegraf Proxmox** collects virtual machine and host metrics from the Proxmox API using the native Telegraf Proxmox input plugin [8]. The plugin authenticates against the Proxmox REST API and retrieves per-VM statistics including CPU utilisation, memory consumption, disk I/O rates, and network throughput for all $VM_1 \dots VM_N$ running on the monitored hosts. Because collection is entirely API-driven and targets the hypervisor management endpoint rather than the guest OS, no monitoring agent needs to be installed on the hypervisor or inside the VMs under test — preserving the measurement integrity of the stress and benchmark workloads running on those VMs.
- **Telegraf Scaphandre** collects power and energy metrics exposed by Scaphandre [12], which reads from hardware interfaces such as Intel RAPL and IPMI. Although power measurement is outside the primary scope of this section, the Scaphandre pod is included in the architecture to illustrate how additional metric sources are integrated: any tool that exposes a Telegraf-compatible input (HTTP, Prometheus scrape, exec, or SNMP) can be wrapped in a dedicated pod and added to the collection pool without modifying the rest of the stack.
- **Telegraf PDU** collects electrical power measurements from rack-mounted Power Distribution Units via SNMP. It exposes per-outlet active power and aggregate rack-level draw, complementing Scaphandre’s software-derived energy estimates with ground-truth electrical measurements at the infrastructure boundary. The SNMP input plugin is vendor-agnostic — any PDU that exposes a standard SNMP MIB requires only a configuration change, not a new container image.
- **Telegraf X** represents the extensibility slot built into the architecture. Adding a new metric source requires only two artefacts: a **container image** packaging the Telegraf configuration for the new input plugin, and a **WorkflowTemplate** definition that tells Argo how to schedule it. The orchestration layer, storage pipeline, and visualisation stack require no modification. Concrete candidates for future extension include:
 - **Containers and microservices** — per-container CPU, memory, and network metrics via the Docker or containerd input plugins.
 - **GPU** — utilisation, memory, temperature, and power draw per card via the `nvidia_smi` plugin.

- **Network-attached devices** — interface counters and error rates from switches and routers via SNMP.
- **Latency** — round-trip and DNS resolution latency via the `ping` and `dns_query` plugins.
- **Storage and NFS** — mount statistics and distributed storage health for shared storage pools.

All Telegraf pods share a common output configuration: collected metrics are tagged with the host identifier, VM name, collection timestamp, and a session run ID supplied by Argo, then forwarded to InfluxDB over the line protocol. The run ID tag enables precise correlation between collection sessions and stress test runs when the two subsystems operate concurrently.

4.1.6 Storage and Visualisation

InfluxDB [9] is the time-series database that persists all metrics emitted by the Telegraf pods. Its data model — organised around measurements, tags, and fields — maps naturally onto the structured, tagged output produced by Telegraf, requiring minimal transformation at ingestion time. Retention policies are configurable per measurement, allowing high-resolution data to be kept for short-term analysis while downsampled aggregates are retained for long-term trend observation.

Grafana [10] connects to InfluxDB as its primary data source and provides the visualisation layer for operators. Dashboards are pre-configured for the principal metric categories — per-VM CPU and memory utilisation, host-level I/O and network throughput, and collection health indicators — and are parameterised by host and VM identifiers so that the same dashboard layout applies across all monitored nodes without modification. When a stress test is running concurrently, Grafana’s annotation feature surfaces the active test profile and its start and end times on the same timeline as the collected metrics, enabling direct visual correlation between applied load and observed system behaviour.

4.1.7 Kafka Integration

Apache Kafka [11] is an optional component that the user may activate through the Argo workflow parameters. When enabled, Telegraf is configured to emit a copy of selected metrics to a Kafka topic in addition to InfluxDB, enabling two complementary use cases:

- **Real-time alerting.** Downstream consumers subscribed to the Kafka topic can apply threshold rules and trigger notifications when metrics exceed defined bounds — for example, when aggregate CPU utilisation across all monitored VMs sustains above a critical level for a configurable duration.
- **Data streaming to external systems.** Kafka acts as a decoupling bus between the Collector Node and any external platform that needs access to the metrics stream — such as a centralised operations dashboard, a capacity planning tool, or a long-term data warehouse — without creating a direct dependency on the Collector Node’s InfluxDB instance.

Because Kafka is activated at the workflow parameter level, it introduces no mandatory dependency on the core collection path. Deployments that do not require streaming or external alerting operate identically with Kafka disabled, keeping the Collector Node footprint minimal.

4.1.8 Scalability and Extensibility

The architecture achieves horizontal scalability through two mechanisms. First, the number of active Telegraf pods scales with the number of hosts and metric sources by adjusting the parallelism argument passed to Argo at submission time — a lightweight deployment runs few pods, a larger environment adds more without architectural changes. Second, the k3s runtime provides fast pod lifecycle management, meaning collection profiles can be started and stopped on-demand with no persistent overhead between sessions.

Table 4.4 summarises the components and their roles.

Table 4.4: Collector Node component summary.

Component	Technology	Role
Orchestrator	Argo Workflows	Schedules and parameterises Telegraf pods
Hypervisor agent	Telegraf (Proxmox)	Collects VM and host metrics via REST API
Power agent	Telegraf (Scaphandre)	Collects energy metrics from RAPL / IPMI
PDU agent	Telegraf (SNMP)	Electrical power per outlet and total rack draw
Extension slot	Telegraf X	Containers, GPU, network devices, latency, storage
Time-series store	InfluxDB	Persists tagged metrics with retention policies
Visualisation	Grafana	Dashboards and run-correlated annotations
Streaming bus	Kafka	Optional alerting and external data streaming
Runtime	k3s	Lightweight Kubernetes for pod lifecycle

4.2 Stress Testing and Performance Benchmarking Architecture

The stress testing subsystem is designed to evaluate the behaviour of a target virtual machine under controlled resource pressure, while simultaneously subjecting a deployed application to realistic HTTP load. The architecture follows a two-node model: a *Test Stress Manager* node that drives orchestration and load generation, and a *VM Under Test* node that receives and executes the workloads.

The containerised stress modules and QuickPizza application were developed by researchers at Instituto de Telecomunicações as part of related projects and are available in the shared repository [18]. The work described in this chapter concerns the orchestration layer, workflow integration, benchmarking pipeline, and the unified user interface built on top of these modules.

Figure 4.2 illustrates the overall topology.

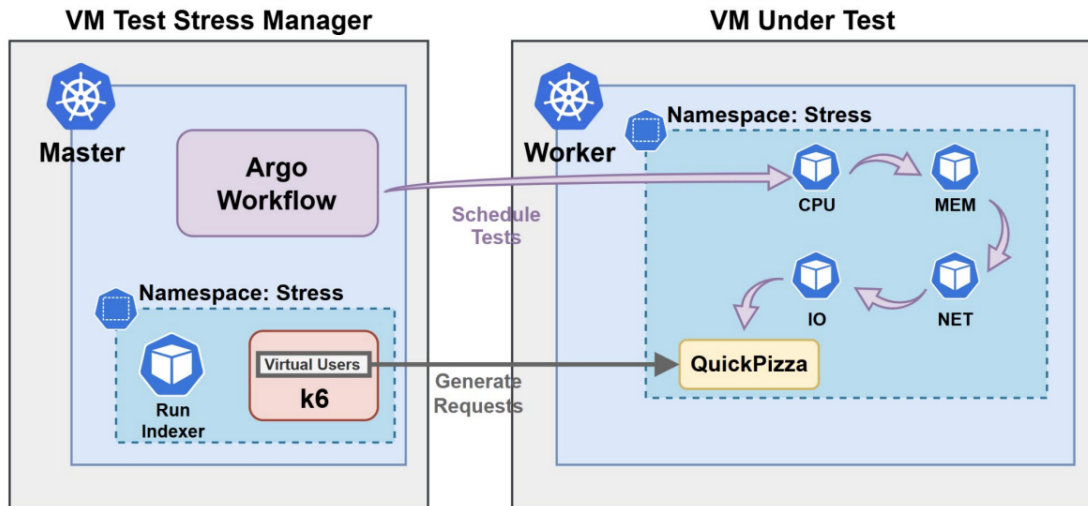


Figure 4.2: Stress testing architecture showing the Test Stress Manager (master) and the VM Under Test (worker), connected through Argo Workflow scheduling and k6 HTTP load generation.

4.2.1 Node Roles

Test Stress Manager — Master Node. The master node is a dedicated virtual machine running a single-node Kubernetes cluster. Its sole responsibility is orchestration and load generation; it does not execute any stress workloads itself. Three components run on this node inside the `stress` namespace:

- **Argo Workflows** acts as the central orchestrator. It exposes a REST API through which test parameters are submitted. When a test run is initiated from the UI, Argo receives the selected `WorkflowTemplate` together with its arguments — such as test type, duration, concurrency level, and target intensity — and translates them into Kubernetes pod scheduling instructions directed at the worker node.
- **k6** is a programmable load testing tool that simulates concurrent virtual users sending HTTP requests to the target application running on the worker. It runs in parallel with the infrastructure stress pods, enabling the measurement of application-level degradation under resource contention.
- **Run Indexer** attaches structured metadata to each test execution — including a unique run identifier, timestamp, selected stress profile, and parameter values. This metadata is propagated to the metrics pipeline so that results from different runs can be correlated and compared on a shared timeline in Grafana.

VM Under Test — Worker Node. The worker node is the virtual machine being evaluated. It joins the same Kubernetes cluster as the master and is labelled so that Argo can target it exclusively for stress pod scheduling. All workload pods are created inside the `stress` namespace on this node. Two categories of workload run here:

- **Infrastructure stress pods** (CPU, MEM, IO, NET) are scheduled by Argo and execute the resource-level workloads described in Section 4.2.3.

- **QuickPizza** is a lightweight web application deployed permanently on the worker. It serves as the HTTP target for k6, providing a realistic application workload that competes with the stress pods for the same underlying resources. The combination of application load and infrastructure stress reveals how resource exhaustion translates into observable application degradation — increased latency, higher error rates, and reduced throughput.

4.2.2 Argo Workflow Orchestration

Argo Workflows [7] is a Kubernetes-native workflow engine that models multi-step jobs as directed acyclic graphs (DAGs) of container pods. Each node in the graph corresponds to a single pod execution; Argo tracks state, handles retries, and propagates parameters between steps.

Test definitions are stored as `WorkflowTemplates` — reusable, parameterised specifications that declare the container image, command arguments, and execution order for each stress type. When a user selects a test from the UI, the interface issues a `POST /api/v1/workflows` request to the Argo server, passing the chosen template name and the parameter values entered in the form. Argo resolves the template and instructs the Kubernetes scheduler to create the corresponding pods on the worker node.

This design cleanly separates concerns: the UI handles human interaction and parameter input, Argo handles scheduling and lifecycle management, and the worker handles execution. Adding a new stress profile or benchmark requires only a new `WorkflowTemplate` definition and a container image; no changes to the UI or the worker configuration are needed.

4.2.3 Stress Test Types

Four infrastructure stress profiles are defined, each targeting a distinct hardware resource:

Table 4.5: Infrastructure stress pod definitions.

Profile	Tool	Primary resource stressed
CPU	<code>stress-ng --cpu</code>	Processor cores and instruction throughput
MEM	<code>stress-ng --memrate</code>	Sequential DRAM read/write bandwidth
IO	<code>fio</code>	Disk read/write throughput and I/O depth
NET	<code>iperf3</code>	Network bandwidth and packet throughput

The DAG structure in the `WorkflowTemplate` controls execution order. Profiles may run sequentially — to isolate the effect of each resource independently — or in parallel to simulate mixed-load scenarios. All profiles accept `duration` and `intensity` parameters that are injected at submission time from the UI.

4.2.4 Benchmarking Phase

Before executing stress tests, a characterisation phase is run once per worker VM to establish a hardware performance baseline across all major subsystems. Five benchmarks are containerised and scheduled as `WorkflowTemplates` within Argo, producing structured JSON results delivered via HTTP callback and tagged with the worker VM identifier and run timestamp. A sixth benchmark, SPEC CPU 2017, is planned pending license acquisition.

4.2.4.1 STREAM — Memory Bandwidth

STREAM [19] measures sustained memory bandwidth across four vector operations: Copy, Scale, Add, and Triad. Results are expressed in MB/s and characterise the physical ceiling of the memory subsystem. The Triad score — which performs a multiply-add across three arrays — is the headline figure used industry-wide for memory subsystem comparison.

This baseline is particularly relevant for interpreting MEM stress results: when the memory stressor saturates bandwidth and QuickPizza latency increases, the STREAM Triad score provides the context for whether the system reached its hardware limit. The benchmark also reports an efficiency percentage when the node's theoretical peak bandwidth is provided, and performs automatic anomaly detection flagging imbalances between operations that may indicate cache or NUMA issues.

4.2.4.2 7-zip — CPU Integer Performance

The 7-zip internal benchmark (7z b) [20] measures CPU integer performance by running compression and decompression workloads at multiple dictionary sizes. Two passes are executed — single-thread and multi-thread — producing MIPS scores for each. The ratio between the two passes yields a scaling efficiency percentage that reveals how effectively the CPU parallelises integer workloads across available cores.

This benchmark complements the CPU stress module: where the stress pod saturates processor cores at a configurable load percentage, the 7-zip benchmark characterises the raw integer throughput available to application workloads before and after stress is applied.

4.2.4.3 iperf3 — Network Performance

iperf3 [14] measures network performance between the worker node and a server running on the master. Four tests are executed in sequence: TCP upload, TCP download, TCP bidirectional, and UDP upload. The TCP tests report throughput in Mbps and retransmit counts; the UDP test additionally reports jitter in milliseconds and packet loss percentage.

4.2.4.4 fio — Storage IO Performance

fio [15] measures storage IO performance across four access patterns: sequential read, sequential write, random read, and random write. Each test uses direct IO (`--direct=1`) to bypass the page cache and measure actual storage device performance. Results include throughput in MB/s, IOPS, and latency percentiles (mean, p50, p95, p99) for each pattern.

The sequential tests characterise bulk transfer capacity relevant to log-intensive workloads, while the random 4K tests characterise IOPS and latency relevant to database-style access patterns. The IO stress module drives the same storage subsystem; the fio baseline establishes the performance envelope within which stress effects are measured.

4.2.4.5 LMBench — Memory Latency

LMBench [21] measures memory access latency across the full cache hierarchy using a pointer-chasing algorithm. Arrays of increasing size are walked sequentially; when the working set fits within L1 cache, access latency is in the range of 1–4 ns. Spilling to L2 raises latency to 5–15 ns, L3 to 30–50 ns, and main memory to 60–150 ns.

The result is a latency curve across array sizes that exposes the cache hierarchy boundaries of the worker VM. This complements STREAM: where STREAM measures bandwidth, LMBench measures latency — together they characterise the memory subsystem completely. The latency curve is returned in the callback payload as a data series suitable for direct rendering as a line chart in the monitoring dashboard.

4.2.4.6 SPEC CPU 2017 — Pending

SPEC CPU 2017 [22] is the industry-standard CPU benchmark used by hardware vendors and cloud providers for processor comparison. It executes a suite of real-world compute workloads — including compilers, physics simulations, and video encoding — producing a composite SPECrate score for throughput across multiple copies.

Implementation is planned but currently blocked pending license acquisition. SPEC CPU 2017 requires a commercial or academic license and cannot be distributed in a public container image; the image will therefore be hosted on a private local registry on the master node, following the same WorkflowTemplate pattern as the other benchmarks.

Table 4.6: Benchmark modules and their target subsystems.

Benchmark	Tool	Subsystem	Key Metrics
STREAM	stream	Memory bandwidth	Copy, Scale, Add, Triad (MB/s)
7-zip	7z b	CPU integer	MIPS; scaling efficiency %
iperf3	iperf3	Network	Throughput (Mbps); jitter (ms); loss %
fio	fio	Storage IO	MB/s; IOPS; latency p50/p95/p99
LMbench	lat_mem_rd	Memory latency	L1/L2/L3/RAM latency (ns)
SPEC CPU 2017	runcpu	CPU int + FP	SPECrate2017 base score

4.3 User Interface Architecture

The user interface is the single point of human interaction with the platform. It bridges the operator and the two backend subsystems — the stress testing cluster described in Section 4.2 and the metrics collection cluster described in Section 4.1 — exposing their configuration surfaces and workflow submission capabilities through a unified web application. Figure 4.3 illustrates the three-component topology and the communication channels that connect them.

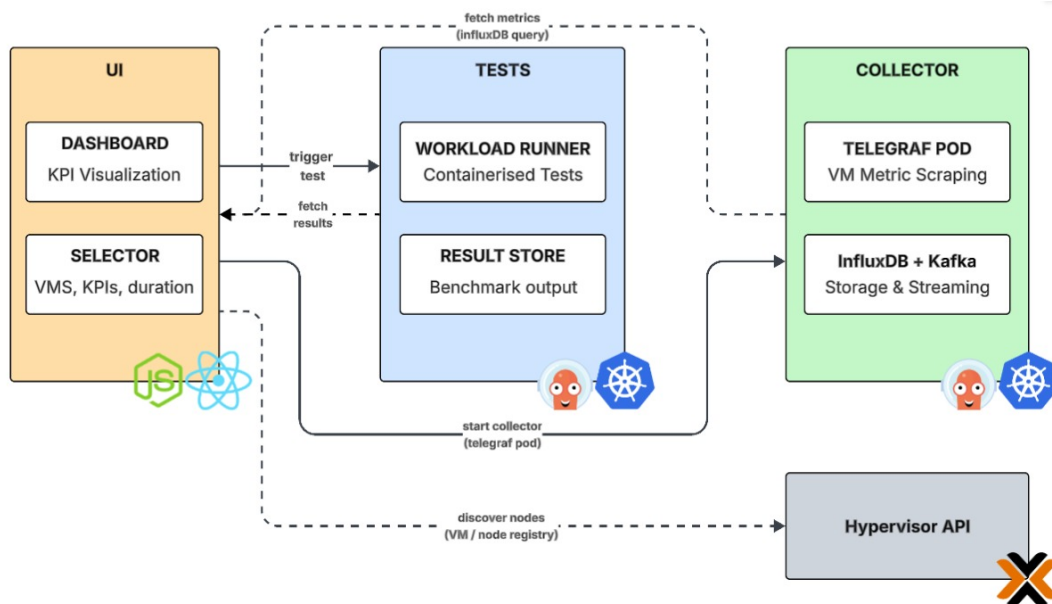


Figure 4.3: Overall system architecture showing the UI, the stress testing cluster (TESTS), and the metrics collection cluster (COLLECTOR), with the four typed communication channels between them.

4.3.1 Components

The frontend is a single-page application built with React [23], served from a dedicated host separate from both Kubernetes clusters. The backend is an Express.js server that forwards requests to the two Argo instances, injects the Bearer token, and handles callback reception. Two functional views are exposed to the operator:

- **Dashboard** — displays collected KPIs fetched from InfluxDB and benchmark results received via callback.
- **Selector** — allows the operator to configure and submit stress tests, benchmarks, and collection workflows, specifying target VMs, KPIs, and duration.

4.3.2 Communication Channels

The UI communicates with the two backend clusters through four typed channels, as shown in Figure 4.3 and summarised in Table 4.7.

Table 4.7: UI communication channels.

Channel	Direction	Purpose
Trigger test	UI → TESTS	Submit a stress or benchmark WorkflowTemplate
Fetch results	TESTS → UI	Retrieve workflow status and benchmark results
Start collector	UI → COLLECTOR	Submit a Telegraf pod collection workflow (via FastAPI)
Discover nodes	UI → HYPERVISOR	Query available hosts and VMs for the node selector

Metric visualisation follows a separate path: the Dashboard component queries InfluxDB directly to retrieve and display collected time-series data, keeping the real-time display independent of the workflow submission flow.

4.3.3 Interaction Flow

A complete test-and-collect session proceeds as follows:

1. The operator discovers available nodes via the Selector and configures the collection session — selecting KPIs, target VMs, and duration.
2. The operator configures and submits a stress test or benchmark. Collection and testing workflows can be submitted independently and can run concurrently to correlate metrics with active stress windows.
3. The UI polls the TESTS cluster for workflow status and receives benchmark results via HTTP callback when tests complete.
4. The Dashboard queries InfluxDB to display the collected metrics, annotated with the stress test window for direct correlation.

4.4 Machine Learning Pipeline — Exploratory Architecture

The stress testing and metrics collection subsystems produce structured, timestamped, run-tagged time-series data as a natural by-product of their operation. This section outlines a possible machine learning pipeline that could exploit this data to provide three additional capabilities: anomaly detection, performance degradation prediction, and bottleneck identification. The architecture described here is exploratory; detailed design and implementation are left for future work.

4.4.1 Overview and Data Sources

Figure 4.4 shows the proposed pipeline at a high level. It follows a Lambda architecture [24], combining a batch path — which trains models from InfluxDB historical data — with a streaming path that serves real-time inference against the live Kafka metric stream.

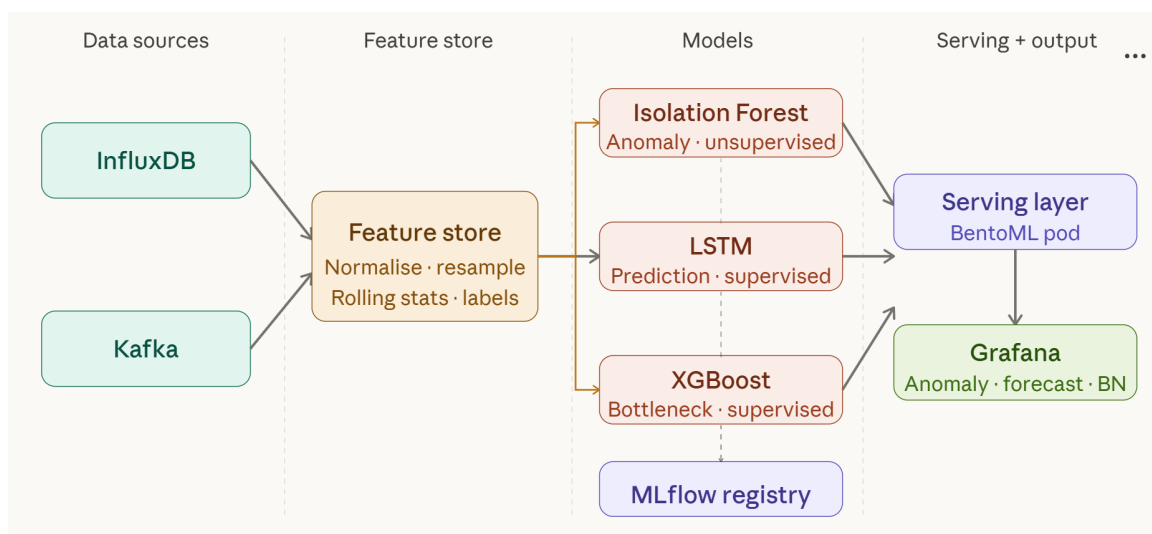


Figure 4.4: High-level ML pipeline overview. Raw metrics from InfluxDB and Kafka are transformed into a shared feature store, used to train three model families, and served to a Grafana dashboard via batch and streaming inference paths.

A key advantage of the existing platform design is that stress test run labels — recording which resource profile was active and when — are already attached to the time-series data via the Run Indexer. This yields a semi-supervised dataset without any manual annotation effort: the stress windows provide ground truth for supervised models, while the remaining periods supply unlabelled normal-behaviour data for unsupervised learning.

4.4.2 Proposed Models

Three model families are proposed, matched to the label availability and nature of each task. Table 4.8 summarises the choices.

Table 4.8: Proposed models per ML task.

Task	Model	Learning type
Anomaly detection	Isolation Forest [25] + LSTM Autoencoder [26]	Unsupervised
Degradation prediction	LSTM forecaster [26]	Supervised
Bottleneck spotting	XGBoost classifier [27]	Supervised

Isolation Forest identifies anomalous metric windows without labels by isolating outliers in the feature space. The LSTM Autoencoder complements it by flagging sequences whose reconstruction error departs from the learned normal pattern, capturing temporal anomalies that point-wise methods may miss. The LSTM forecaster predicts key metric values N steps ahead, enabling early warning of impending saturation. XGBoost classifies the binding resource constraint at each time step and produces interpretable feature importance scores, allowing the operator to understand which specific metrics drove the classification.

All training jobs are orchestrated by the existing Argo Workflows installation and tracked by **MLflow** [28], which maintains a versioned model registry and enforces a validation gate before promoting a new model to production.

4.4.3 Inference and Output

Figure 4.5 shows the two inference paths.

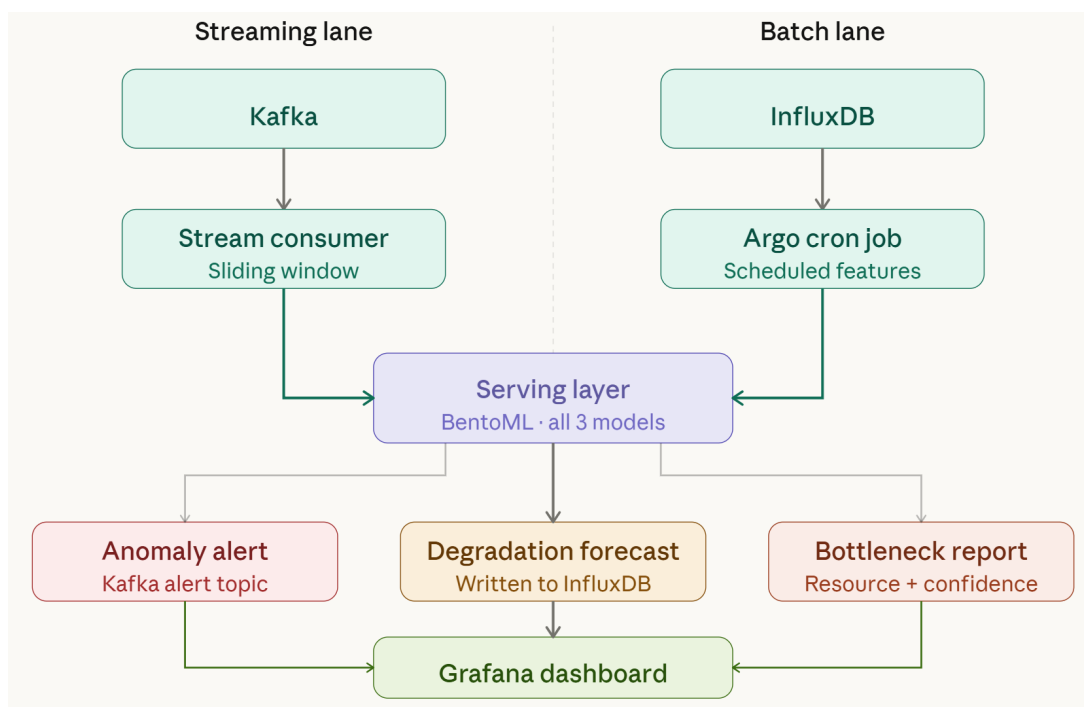


Figure 4.5: Inference paths. The streaming lane produces near-real-time anomaly alerts; the batch lane produces degradation forecasts and bottleneck reports. Both write results to InfluxDB for visualisation in Grafana.

Models are served via **BentoML** [29] deployed as a Kubernetes pod alongside the existing collector components. Outputs are written back to InfluxDB and surfaced in Grafana through three new panels: a continuous anomaly score overlay, a degradation forecast band, and a per-VM bottleneck heatmap. Stress test run annotations are displayed on all panels, enabling direct visual correlation between applied load and model outputs. Retraining is triggered automatically on each new stress test run completion, so model accuracy improves continuously as the test dataset grows.

CHAPTER 5

Results

This chapter describes the results for the development of this project, based on the already defined objectives. The main focus of our work lies in the implementation and validation of a functional system capable of monitoring, analyzing and testing environments in dynamic datacenters.

5.1 MVP - Minimum Viable Product

The MVP is a development technique, that consist of building only the defined minimal, basic features that are enough to get customers interested. In this projects case, the mvp was defined has these features:

- Multi-layer metric collection, allowing relevant data to be gathered;
- Monitoring tool capable of storing and deposing data at real-time;
- Easy automated execution of stress testing and fault ejections, allowing simulations of real world adversities;
- Dashboard for easy visualisation enabling the user to follow the status of the system and intuitive interaction, allowing stress test and fault ejection configuration;

5.2 Observability and Monitoring Capabilities

The project achieved a significant improvement in the observability of monitoring system environments. This includes:

- The autonomous and centralized gathering of relevant metrics;
- The capability to correlate event between different components;
- Real-time integrated view of the system status.

These improvements enhanced the capability to identify patterns of behavior and facilitated the analysis of the status of distributed environments.

5.3 Testing and Validation Framework

A central contribution of this project lies in the implementation of testing mechanisms directly within the monitoring framework. This integration allowing:

- Stress Testing, evaluating system behavior and stability under varying levels of demand;
- Impact analysis, assessing the consequences of failures by analyzing metrics gathered during active testing.
- Benchmark Performance Testing, evaluating core infrastructure through real-world market-oriented simulations;

By testing the system under near-real-world conditions, this approach provides a much clearer understanding of its overall resilience.

5.4 Dashboard

The platform's user interface was implemented as a unified web application providing a single point of interaction for all platform subsystems. The following sections describe each functional view and its observed behaviour during validation.

5.4.1 Initial Pages and Overview

Upon accessing the platform, the operator is presented with a login page requiring institutional credentials. Registration is restricted to addresses belonging to the @av.it.pt domain, ensuring access is limited to authorised personnel. After authentication, the operator can update their password through the user profile page, accessible via the username in the navigation bar.

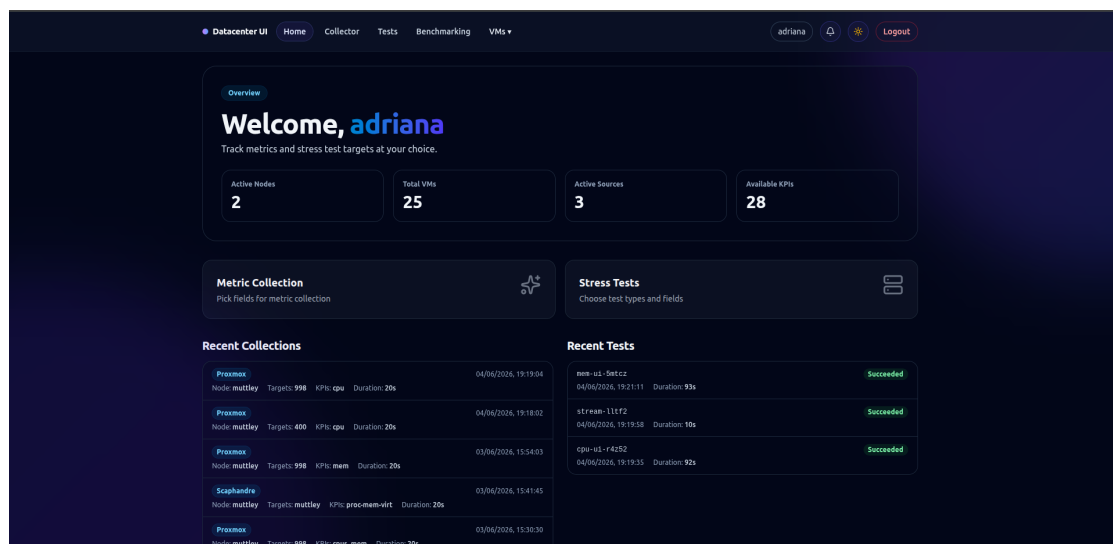


Figure 5.1: Home page

The Home page provides a personalised overview of the platform state. Four summary cards display the number of Active Nodes, Total VMs, Active Sources, and Available KPIs. Below, two quick-access shortcuts link directly to the Metric Collection and Stress Tests pages.

The lower half of the page presents two activity feeds: Recent Collections, listing the latest collection sessions with source, node, targets, KPIs, duration, and timestamp; and Recent Tests, showing the most recently submitted stress tests.

The notification system is accessible via the bell icon in the navigation bar, which displays a badge with the number of unread alerts. The panel lists events in chronological order, covering the full workflow lifecycle.

5.4.2 Metrics Collection

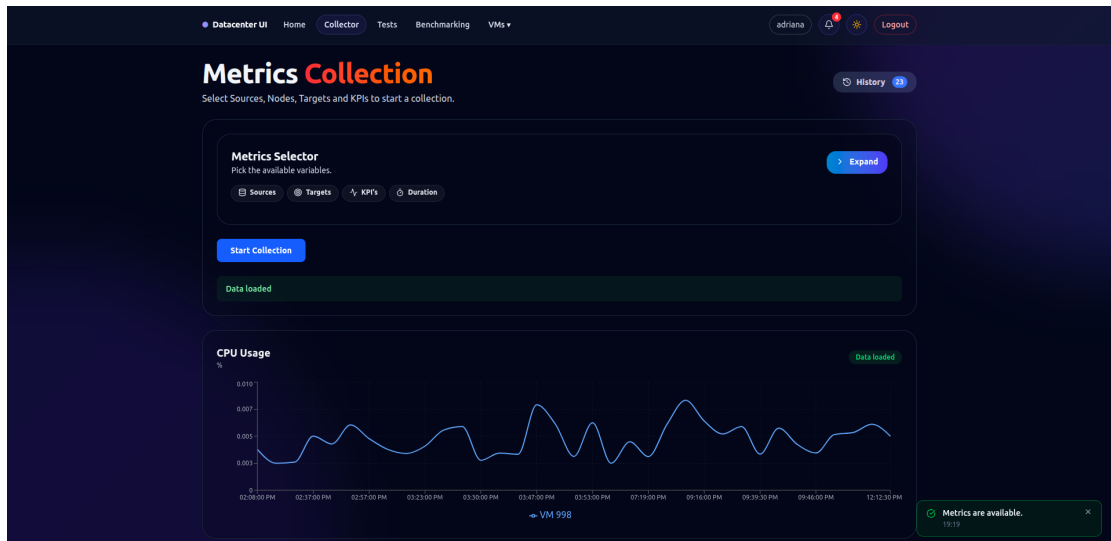


Figure 5.2: Collector page

The Metrics Collection page allows the operator to configure and launch a collection session. The Metrics Selector panel guides the operator through a source-dependent configuration flow. The operator first selects the data source — Proxmox, Scaphandre, or PDU — after which the remaining fields adapt accordingly. Once configured, the operator clicks Start Collection to submit the request. The collected time-series data is displayed directly below the selector as an interactive chart per KPI, labelled by VM identifier.

Each completed session is recorded and accessible via the History panel. The operator can click any item to open a detail view displaying the session metadata alongside the corresponding time-series chart. Sessions can be individually selected or bulk-selected for export as a CSV file, allowing offline analysis of collected metrics.

Table 5.1: Collector configuration parameters.

Source	Parameter	Type	Description
All	source	string	Data source — proxmox, scaphandre, pdu
	duration	int (s)	Collection window duration per KPI job. Default: 30 s
Proxmox	host	string	Proxmox API host IP
	node	string	Proxmox node name (e.g. pve)
	target	string	VM ID to collect metrics from
Scaphandre	host	string	Scaphandre exporter IP
	target	string	Machine name label
PDU	host	string	PDU SNMP IP

5.4.3 Stress Tests

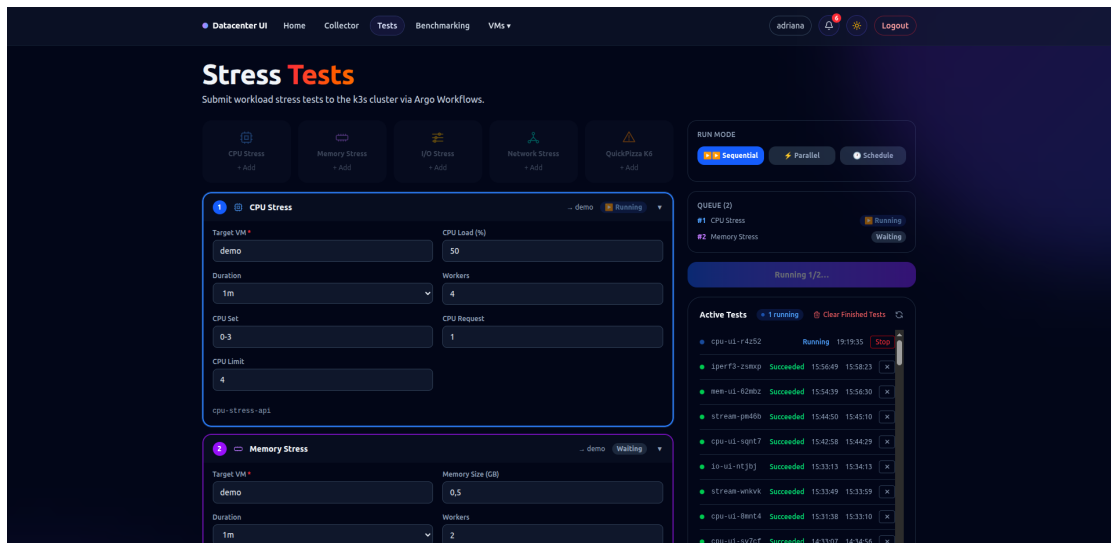


Figure 5.3: Stress Tests page

The Stress Tests page provides a queue-based interface for submitting workload stress tests to the k3s cluster via Argo Workflows. The operator selects one or more test types — CPU Stress, Memory Stress, I/O Stress, Network Stress, and QuickPizza K6 — and configures each with its specific parameters. Tests are added to a numbered queue and executed according to the selected run mode: Sequential, Parallel, or Schedule. The Active Tests panel displays real-time workflow status with start and end timestamps, and allows running tests to be stopped manually.

For stress tests other than QuickPizza, result inspection is performed externally via the Proxmox management interface, which provides per-VM resource graphs. For QuickPizza K6 tests, results are surfaced directly in the UI upon completion, displaying the test results alongside the raw k6 JSON output.

Table 5.2: Stress test input parameters.

Module	Parameter	Description
All	target-node	Worker node hostname
	callback-url	HTTP endpoint to receive result payload
CPU	workers	Number of stress-ng CPU workers
	load	CPU load percentage per worker (1-100%)
	timeout	Duration (e.g. 30s, 1m)
	cpuset	CPU cores to pin workers to (e.g. 0-3)
Memory	workers	Number of memory stress workers
	size	Memory per worker in GB
	timeout	Duration
	read_mbps	Read throttle in MB/s (-1 = unlimited)
	write_mbps	Write throttle in MB/s (-1 = unlimited)
IO	mode	Access pattern — seq-read, seq-write, rand-read, rand-write
	runtime_sec	Duration in seconds
	target_mbps	Target throughput in MB/s
	block_size	IO block size (e.g. 4K, 1M)
	iodepth	IO queue depth
Network	server	IP of the iperf3 server
	bandwidth	Target bandwidth in Mbps
	proto	Protocol — tcp or udp
	direction	upload or download
	time	Duration in seconds
QuickPizza K6	run_id	Unique identifier for the benchmark execution
	base_url	QuickPizza API endpoint inside the cluster
	max_vusers	Maximum concurrent virtual users
	duration	Test duration in seconds
	runner_count	Number of parallel k6 runner pods

5.4.4 Benchmarking

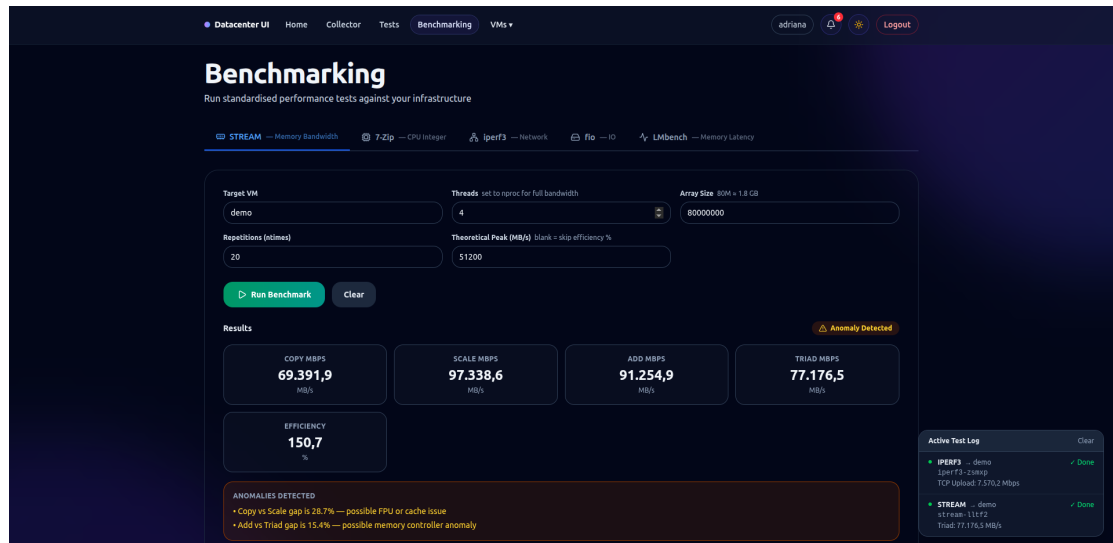


Figure 5.4: Benchmarking page

The Benchmarking page runs standardised performance tests against the infrastructure. Five benchmark tools are available as tabs: STREAM, 7-Zip, iperf3, fio, and LMbench. Each tool exposes its specific parameters. Results are displayed immediately after execution as labelled metric cards. The backend analyses the results and flags anomalies automatically: for example, in the validation run shown, two anomalies were detected in the STREAM results. The Active Test Log confirms concurrent benchmark execution across multiple tools.

Table 5.3: Benchmark input parameters.

Benchmark	Parameter	Description
All	target-node	Worker node hostname
	callback-url	HTTP endpoint to receive result payload
STREAM	threads	OMP_NUM_THREADS — set to nproc for full bandwidth
	array_size	Elements per array (80 M $\approx$ 1.8 GB)
	ntimes	Number of passes — best of N is reported
	theoretical_peak_mbps	Node theoretical RAM peak for efficiency % (optional)
7-zip	threads	Thread count for multi-thread pass
iperf3	server	IP of the iperf3 server
	time	Duration per test in seconds (4 tests total)
	parallel	Number of parallel TCP/UDP streams
	udp_bandwidth	UDP send rate in Mbps
fio	file_size	Test file size — must exceed RAM (default: 2 G)
	runtime	Seconds per test (4 tests total)
	iodepth	IO queue depth
	numjobs	Parallel fio worker threads
LMbench	stride	Pointer jump distance in bytes (default: 128)
	max_size	Largest array size to test (default: 256 M)

5.4.5 VM Dashboard

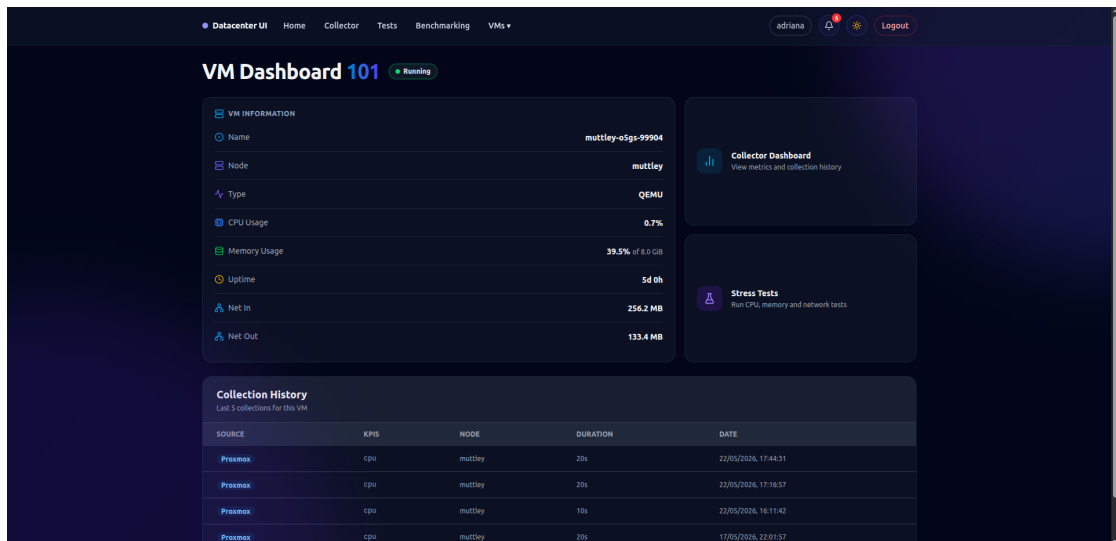


Figure 5.5: VM Dashboard page

The VM Dashboard provides a per-VM operational view, displaying relevant operational information and resource usage. The Collection History table lists the most recent collection sessions for that VM, showing source, KPIs, node, duration, and timestamp. Two quick-access shortcuts link directly to the Collector Dashboard and the Stress Tests page.

5.5 Documentation and Deliverables

Ultimately, the definitive deliverables of this project consist of the following components:

- This comprehensive technical report detailing the system requirements, methodologies, and findings;
- Detailed technical documentation of the developed architectures and software systems including API references for the stress, benchmark, and collector modules;
- A fully functional prototype deployed and validated within a production-ready environment;
- A dedicated project website providing an accessible, high-level, and scannable overview of the project's lifecycle and outcomes [30].

Conclusions & Future Work

6.1 Conclusions

6.1.1 Summary of Platform Capabilities

Following the detailed analysis of individual testing components, this section contextualizes how these modules coalesce into a unified evaluation framework. The platform provides a single user interface designed to streamline the operational lifecycle, allowing users to submit test configurations, monitor ongoing execution, and retrieve comprehensive telemetry datasets from a centralized dashboard.

To deliver complete multi-layered observability, the framework aggregates a matrix of **28 distinct metrics** collected continuously across three structural layers:

- **Hypervisor Layer (Proxmox):** Capitalizes on native virtualization telemetry to track resource allocation and guest VM behaviors.
- **Software Energy Layer (Scaphandre):** Obtains high-fidelity, software-defined power utilization metrics mapped directly to process workloads.
- **Hardware Infrastructure Layer (PDUs):** Interfaces directly with physical Power Distribution Units to record definitive, hardware-level power draw.

This monitoring fabric operates concurrently with the platform's execution suites, which comprise the five stress modules (CPU, Memory, IO, Network, and QuickPizza for HTTP load testing) and the five performance benchmark modules (STREAM, 7-zip, iperf3, fio, and LMBench) detailed previously in this work.

6.1.2 System Modularity and Integration Principles

The core architectural value of the framework lies in its strict enforcement of modular decoupling. The primary design objective ensures that the core orchestration engine, underlying database storage schemas, and the front-end user interface remain entirely agnostic of the specific stress or benchmarking logic executed at the node level.

Consequently, expanding the system's capabilities—whether by integrating a new synthetic workload, an industry-standard benchmark suite, or a telemetry data source—requires zero modifications to the foundational architecture. This identical design philosophy governs both underlying systems within the platform, relying on a declarative, self-contained implementation pattern. To introduce a new module or data collector, an operator must only provision two assets:

1. **A Standardized Container:** A container image enclosing the execution runtime, libraries, and binaries of the target test tool or exporter, ensuring complete environment isolation.
2. **A Structured Definition File:** A workflow template, YAML, specifying execution arguments, environmental variables, dashboard mappings, and metadata schemas required by the core engine.

By abstracting backend complexities through this containerized plugin model, both parallel systems are seamlessly integrated into a single, cohesive user experience. Operators can orchestrate heterogeneous test schedules, visualize multi-source metric streams, and process analytical outputs in real-time without introducing technical debt or architectural instability to the platform.

6.2 Future Work

6.2.1 Intelligent Analysis

The platform is designed with ML readiness in mind — every run is labelled, timestamped, and structured — but the dataset required for model training does not yet exist. The immediate next step is dataset formation: defining what constitutes normal performance, and establishing ground truth labels for anomalous and failure conditions from accumulated stress run history. With a sufficient dataset, three ML tasks become viable: anomaly detection, performance degradation prediction, and bottleneck identification.

6.2.2 Extended Metrics Collection

The Telegraf plugin model makes the following sources straightforward additions, each requiring only a new container image and a `WorkflowTemplate`: containers and microservices, GPU utilisation and power, network-attached devices, memory and network latency, and storage allocation including NFS.

6.2.3 Chaos Engineering and Realistic Failure Scenarios

The current stress modules provide controlled, parameterised resource pressure — CPU load percentage, memory allocation rate, IO depth, and network bandwidth — which gives the operator precise control over the conditions applied. True chaos engineering, as understood in the literature [31], involves fault injection into real running services: process kills, network partitions, latency injection, and cascading failure scenarios derived from actual production workload profiles. Extending the platform in this direction would complement the existing parameterised approach and enable the study of failure propagation across service dependencies.

6.2.4 Security and Dashboard Improvements

The dashboard currently has no access control. Future work includes a role-based access control (RBAC) system with two roles — **Administrator** and **User**. Administrators would control persistent infrastructure configuration: collector endpoints, worker nodes, and data source credentials. Users would be restricted to operational tasks: submitting tests, triggering collections, and viewing results. This separation protects infrastructure integrity in shared research environments where multiple operators share the same platform. A persistent user database and authentication layer would underpin both roles.

On the visualisation side, the dashboard would benefit from richer statistical analysis — distribution views, cross-run comparisons, and baseline overlays. Aggregated views across

multiple VMs and nodes would further enable operators to inspect summed or averaged resource consumption at the cluster level, complementing the existing per-VM granularity.

6.2.5 Automation

Several operational tasks currently require manual intervention and are candidates for automation using Ansible [32] automation: provisioning new performance testing nodes into the k3s cluster, and configuring collection agents that require access inside the VM — such as installing latency probes or deploying Scaphandre on a new host. These tasks involve direct VM interaction that sits outside the current containerised workflow model and would benefit from automated playbooks to maintain consistency and reduce onboarding time for new nodes or sources.

Bibliography

- [1] IMAGINE-B5G Consortium, *IMAGINE-B5G: Advanced 5G open platform for large scale trials and pilots across europe*, 2022. [Online]. Available: <https://imagineb5g.eu>
- [2] EXIGENCE Consortium, *EXIGENCE: Devise & explore a novel approach for energy consumption and carbon footprint reduction of ICT services in the era of next-generation mobile telecommunications (6G)*, 2024. [Online]. Available: <https://projectexigence.eu>
- [3] 6G-VERSUS Consortium, *6G-VERSUS: 6g vertical trials for sustainability*, 2024. [Online]. Available: <https://6g-versus.eu>
- [4] Proxmox Server Solutions GmbH, *Proxmox Virtual Environment*, 2024. [Online]. Available: <https://www.proxmox.com/en/proxmox-virtual-environment>
- [5] The Kubernetes Authors, *Kubernetes*, 2014. [Online]. Available: <https://kubernetes.io>
- [6] Rancher Labs, *K3s: Lightweight Kubernetes*, 2024. [Online]. Available: <https://k3s.io>
- [7] Argo Project, *Argo Workflows — the workflow engine for Kubernetes*, 2024. [Online]. Available: <https://argoproj.github.io/workflows>
- [8] InfluxData, *Telegraf Proxmox input plugin*, 2024. [Online]. Available: <https://github.com/influxdata/telegraf/tree/master/plugins/inputs/proxmox>
- [9] InfluxData, *InfluxDB: Time series data platform*, 2024. [Online]. Available: <https://www.influxdata.com>
- [10] Grafana Labs, *Grafana: The open observability platform*, 2024. [Online]. Available: <https://grafana.com>
- [11] Apache Software Foundation, *Apache Kafka: A distributed streaming platform*, 2024. [Online]. Available: <https://kafka.apache.org>
- [12] Hubblo, *Scaphandre: Energy consumption metrology agent*, 2024. [Online]. Available: <https://github.com/hubblo-org/scaphandre>
- [13] Chaos Mesh Authors, *Chaos mesh*, 2019. [Online]. Available: <https://chaos-mesh.org>
- [14] ESnet and the iPerf3 Contributors, *Iperf3*, 2014. [Online]. Available: <https://iperf.fr>
- [15] J. Axboe, *Fio — flexible i/o tester*. [Online]. Available: <https://fio.readthedocs.io>
- [16] T. Barros, *Module stresser*, 2025. [Online]. Available: <https://github.com/vlanx/module-stresser>
- [17] OpenStack Foundation, *OpenStack: Open source cloud computing infrastructure*, 2024. [Online]. Available: <https://www.openstack.org>
- [18] Tiago Barros and Alisson Chaves, *Module-stresser*, <https://github.com/vlanx/module-stresser>, Project repository, 2026.
- [19] J. D. McCalpin, *STREAM: Sustainable memory bandwidth in high performance computers*, 1995. [Online]. Available: <https://www.cs.virginia.edu/stream/>

- [20] I. Pavlov, *7-zip: Internal benchmark*, Accessed: 2026, 2016. [Online]. Available: <https://www.7-zip.org/>
- [21] L. McVoy and C. Staelin, *Lmbench: Tools for performance analysis*. [Online]. Available: <http://www.bitmover.com/lmbench/>
- [22] Standard Performance Evaluation Corporation, *SPEC CPU 2017*, <https://www.spec.org/cpu2017/>, Accessed: 2026, 2017.
- [23] Meta Open Source, *React: The library for web and native user interfaces*, 2024. [Online]. Available: <https://react.dev>
- [24] N. Marz and J. Warren, *Big Data: Principles and Best Practices of Scalable Real-Time Data Systems*. Manning Publications, 2015.
- [25] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest", in *Proceedings of the IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.
- [26] S. Hochreiter and J. Schmidhuber, "Long short-term memory", *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [27] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system", in *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [28] Databricks, *MLflow: An open source platform for the machine learning lifecycle*, 2024. [Online]. Available: <https://mlflow.org>
- [29] BentoML Inc., *BentoML: The unified model serving framework*, 2024. [Online]. Available: <https://bentoml.com>
- [30] A. Silva, A. Rocha, D. Oliveira, G. Goulart, and R. Ferreira, *Modular Monitoring and Performance Benchmarking in Virtualised Infrastructures — Project Website*, https://rafa2025.github.io/Site_PECI/, Accessed: June 2026, 2026.
- [31] A. Basiri et al., "Chaos engineering", *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.
- [32] Red Hat, Inc., *Ansible*, 2012. [Online]. Available: <https://www.ansible.com>